

HPC Assignment 3 — Parallel Bubble Sort using OpenMP

This assignment implements:

Sequential and Parallel Bubble Sort

using:

- OpenMP
- odd-even transposition method
- performance comparison

1. What We Have To Do?

We have to:

1. input array elements
2. sort array sequentially
3. sort array using parallel OpenMP
4. compare execution time

Main Goal

To demonstrate:

how parallelism improves sorting performance

2. What is Bubble Sort?

Bubble Sort is a sorting algorithm where:

adjacent elements are repeatedly compared and swapped

until array becomes sorted.

Example

Array:

5 3 1 4

After sorting:

1 3 4 5

Why Called Bubble Sort?

Because:

- larger elements “bubble” toward end after each pass.

3. Sequential Bubble Sort

Code:

```
void sequentialBubbleSort(vector<int> &arr)
```

Working

For every iteration:

- compare adjacent elements
- swap if wrong order

Example Iteration

Array:

5 3 1 4

Pass 1

Compare:

5 and 3

Swap:

3 5 1 4

Compare:

5 and 1

Swap:

3 1 5 4

Compare:

5 and 4

Swap:

3 1 4 5

Largest element reaches end.

4. Sequential Bubble Sort Code Explanation

Outer Loop

```
for (int i = 0; i < n - 1; i++)
```

Controls:

number of passes

Need:

n-1 passes

Inner Loop

```
for (int j = 0; j < n - i - 1; j++)
```

Compares adjacent elements.

Why n-i-1 ?

After each pass:

- largest element fixed at end
- no need to compare again

Swap Condition

```
if (arr[j] > arr[j + 1])
```

Wrong order check.

Swap

```
swap(arr[j], arr[j + 1]);
```

Exchanges elements.

5. Parallel Bubble Sort

MOST IMPORTANT PART.

Code:

```
void parallelBubbleSort(vector<int> &arr)
```

Problem with Normal Bubble Sort Parallelization

Normal bubble sort cannot be directly parallelized because:

swaps may overlap

Example:

5 4 3

Comparing:

- (5,4)
- (4,3)

simultaneously causes conflicts.

Solution → Odd-Even Transposition Sort

Parallel version uses:

odd-even phases

6. Even Phase

Code:

```
#pragma omp parallel for  
for (int j = 0; j < n - 1; j += 2)
```

What Happens?

Compare pairs:

(0,1)

(2,3)

(4,5)

These pairs do NOT overlap.

So:

safe parallel execution

Example

Array:

5 3 8 2

Compare simultaneously:

- (5,3)
- (8,2)

After swaps:

3 5 2 8

7. Odd Phase

Code:

```
#pragma omp parallel for
```

```
for (int j = 1; j < n - 1; j += 2)
```

What Happens?

Compare:

(1,2)

(3,4)

Again:

non-overlapping pairs

Example

From:

3 5 2 8

Compare:

- (5,2)

Swap:

3 2 5 8

Why Odd-Even Method Works?

Because:

- simultaneous swaps never touch same elements
- avoids race conditions

8. Why #pragma omp parallel for Used?

Converts loop into:

parallel execution

Different threads process different pairs simultaneously.

Example

Thread 1:

compare (0,1)

Thread 2:

compare (2,3)

at same time.

9. Main Function

```
int main()
```

Program starts here.

Input Array

```
vector<int> arr(n), arr_copy;
```

Two arrays:

- one for sequential
- one for parallel

Why Copy Needed?

Both algorithms should sort:

same original data

for fair timing comparison.

10. Timing Sequential Execution

```
double start = omp_get_wtime();  
Records start time.
```

Run Sequential Sort

```
sequentialBubbleSort(arr);
```

End Time

```
double end = omp_get_wtime();
```

Total Time

```
(end - start)  
Execution duration.
```

11. Timing Parallel Execution

Same process repeated for:

parallel bubble sort

12. Why Parallel Faster?

Because:

- multiple comparisons happen simultaneously
- multicore CPU utilized

13. Example Execution

Input:
5 3 1 4 2

Sequential Result

1 2 3 4 5

Parallel Result

1 2 3 4 5

Same output.

But:

parallel time usually smaller

14. Time Complexity

Sequential Bubble Sort:
 $O(n^2)$

Parallel Bubble Sort

Approximate:

$O\left(\frac{n^2}{P}\right)$

Where:

- (P) = processors/threads

Why Still (n^2)?

Because:

- total comparisons still quadratic
- only distributed among threads

15. Important Viva Questions

Q1. What is Bubble Sort?

Sorting algorithm using repeated adjacent swaps.

Q2. Why Bubble Sort difficult to parallelize directly?

Because adjacent swaps may overlap.

Q3. What is Odd-Even Transposition Sort?

Parallel variation of bubble sort using even and odd comparison phases.

Q4. Why even and odd phases used?

To avoid overlapping swaps and race conditions.

Q5. What does `#pragma omp parallel` for do?

Executes loop iterations in parallel.

Q6. Why `omp_get_wtime()` used?

To measure execution time.

Q7. Why array copy used?

To compare sequential and parallel sorting fairly.

Q8. What is time complexity of sequential bubble sort?

$O(n^2)$

Q9. What is time complexity of parallel bubble sort?

$O\left(\frac{n^2}{P}\right)$

16. Short Practical Explanation

This program implements sequential and parallel bubble sort using OpenMP. The parallel version uses odd-even transposition sort, where non-overlapping adjacent pairs are compared simultaneously using `#pragma omp parallel` for. Execution times of sequential and parallel sorting are compared using `omp_get_wtime()`.